TRD

Technical Requirements Document (TRD)

AWS Glue ETL Pipeline for Customer Order Processing

-

1. Document Overview

Purpose

This Technical Requirements Document provides implementation-ready technical specifications for an AWS Glue-based ETL pipeline that processes customer and order data. The TRD translates functional requirements into concrete technical design elements including data schemas, transformation logic, AWS Glue job configurations, Apache Hudi table specifications, and error handling strategies.

Scope

This TRD covers technical implementation details for:

- AWS Glue job configuration and runtime parameters
- CSV file ingestion from S3 using Glue DynamicFrame APIs
- Data quality transformations (null removal, deduplication)
- AWS Glue Data Catalog table registration with schema definitions
- Apache Hudi table implementation for SCD Type 2 processing
- Incremental processing logic using Hudi upsert operations
- Aggregation logic and output table generation
- CloudWatch logging and error handling strategies
- Athena query compatibility requirements

This TRD does NOT cover:

- Infrastructure provisioning (IAM roles, S3 buckets, VPC configuration)
- CI/CD pipeline setup
- Cost optimization strategies
- Performance tuning beyond basic recommendations
- Data lineage tracking implementation
- Backup and recovery procedures

Assumptions

1. AWS Glue 3.0 or higher with Python Shell or Spark engine
2. Apache Hudi 0.10.0+ libraries available in Glue environment

3. S3 bucket `adif-sdlc` exists with appropriate folder structure

4. Glue database `gen_ai_poc_databrickscoe` is pre-created

5. IAM role attached to Glue job has permissions for S3 read/write, Glue Catalog operations, and CloudWatch Logs

6. CSV files use comma delimiter with header row

7. Date fields in Order dataset are in ISO 8601 format (YYYY-MM-DD) or parseable string format

8. CustId is string type and serves as join key between Customer and Order datasets

9. Numeric fields (PricePerUnit, Qty) are decimal-compatible

10. Hudi table format is COW (Copy-On-Write) for Athena compatibility

11. System-generated timestamps use UTC timezone

12. Change detection for incremental processing uses full dataset comparison (no CDC mechanism)

13. Glue job runs in a single execution context (no distributed job orchestration)

14. Target S3 paths support overwrite or append operations as specified per requirement

15. Athena workgroup has Hudi connector enabled for querying Hudi tables

•

2. Technical Requirements

TR-INGEST-001: Ingest Customer Data from S3

• Related Functional Requirement ID(s): FR-INGEST-001

• Objective:

Read customer CSV files from S3 source location into AWS Glue DynamicFrame for downstream processing.

• Source Details:

• Source Type: S3 CSV Files

• Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/

• File Format: CSV with header

• Delimiter: Comma (,)

• Encoding: UTF-8 (assumed)

• Target Details:

• Target Type: In-memory Glue DynamicFrame

• Target Name: customer_raw_df

• Input Schema:

Not enough information available in the FRD to derive complete schema details for customerdata source. Based on FRD, the following columns are present:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	(not specified)	(not specified)	(not specified)	
	Name	(not specified)	(not specified)	(not specified)	
	EmailId	(not specified)	(not specified)	(not specified)	
	Region	(not specified)	(not specified)	(not specified)	

• Output Schema:

Same as Input Schema (no transformation at this stage)

• Processing / Transformation Logic:

1. Use `glueContext.create_dynamic_frame.from_options()` to read CSV files

2. Specify format options:

• format="csv"

• withHeader=True

• separator=","

3. Read all files matching pattern in source path

4. Load into DynamicFrame named `customer_raw_df`

5. No schema enforcement or type casting at this stage

• Validation Rules:

• Source path must exist and be accessible

• At least one CSV file must be present in source path

• CSV files must have header row matching expected column names

• Files must be readable and parseable as CSV

• Error Handling and Logging:

• Log start of ingestion with source path and timestamp

• If S3 path does not exist: Log error "Source path s3://adif-sdlc/sdlc_wizard/customerdata/ not found" and raise exception to fail job

• If no CSV files found: Log error "No CSV files found in source path" and raise exception to fail job

• If CSV parsing fails: Log error with file name and parsing exception details, then raise exception to fail job

• Log successful ingestion with record count

• All logs written to CloudWatch Logs group `/aws-glue/jobs/`

• Runtime Parameters:

• `--SOURCE_CUSTOMER_PATH`: s3://adif-sdlc/sdlc_wizard/customerdata/ (default)

• `--CSV_DELIMITER`: , (default)

• `--CSV_HEADER`: True (default)

• Operational Notes:

• Ingestion should handle multiple CSV files in source path

• No schema validation against Glue Catalog at this stage

• Performance: Use Glue DPU allocation appropriate for file size (2-5 DPUs for small datasets)

•

TR-INGEST-002: Ingest Order Data from S3

• Related Functional Requirement ID(s): FR-INGEST-002

• Objective:

Read order CSV files from S3 source location into AWS Glue DynamicFrame for downstream processing.

• Source Details:

• Source Type: S3 CSV Files

• Source Path: s3://adif-sdlc/sdlc_wizard/orderdata/

• File Format: CSV with header

• Delimiter: Comma (,)

• Encoding: UTF-8 (assumed)

• Target Details:

• Target Type: In-memory Glue DynamicFrame

• Target Name: order_raw_df

• Input Schema:

Not enough information available in the FRD to derive complete schema details for orderdata source. Based on FRD, the following columns are present:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	(not specified)	(not specified)	(not specified)	
	ItemName	(not specified)	(not specified)	(not specified)	
	PricePerUnit	(not specified)	(not specified)	(not specified)	
	Qty	(not specified)	(not specified)	(not specified)	

	Date	(not specified)	(not specified)	(not specified)	
	CustId	(not specified)	(not specified)	(not specified)	

• Output Schema:

Same as Input Schema (no transformation at this stage)

• Processing / Transformation Logic:

1. Use `glueContext.create_dynamic_frame.from_options()` to read CSV files

2. Specify format options:

• format="csv"

• withHeader=True

• separator=","

3. Read all files matching pattern in source path

4. Load into DynamicFrame named `order_raw_df`

5. No schema enforcement or type casting at this stage

• Validation Rules:

• Source path must exist and be accessible

• At least one CSV file must be present in source path

• CSV files must have header row matching expected column names

• Files must be readable and parseable as CSV

• Error Handling and Logging:

• Log start of ingestion with source path and timestamp

• If S3 path does not exist: Log error "Source path s3://adif-sdlc/sdlc_wizard/orderdata/ not found" and raise exception to fail job

• If no CSV files found: Log error "No CSV files found in source path" and raise exception to fail job

• If CSV parsing fails: Log error with file name and parsing exception details, then raise exception to fail job

• Log successful ingestion with record count

• All logs written to CloudWatch Logs group `/aws-glue/jobs/`

• Runtime Parameters:

• `--SOURCE_ORDER_PATH`: s3://adif-sdlc/sdlc_wizard/orderdata/ (default)

• `--CSV_DELIMITER`: , (default)

• `--CSV_HEADER`: True (default)

• Operational Notes:

• Ingestion should handle multiple CSV files in source path

• No schema validation against Glue Catalog at this stage

• Performance: Use Glue DPU allocation appropriate for file size (2-5 DPUs for small datasets)

•

TR-CLEAN-001: Remove Null Values from Customer Dataset

• Related Functional Requirement ID(s): FR-CLEAN-001

• Objective:

Remove all records from customer dataset that contain string literal "Null" or actual NULL values in any column.

• Source Details:

• Source Type: In-memory Glue DynamicFrame

• Source Name: customer_raw_df (from TR-INGEST-001)

• Target Details:

• Target Type: In-memory Glue DynamicFrame

• Target Name: customer_clean_nulls_df

• Input Schema:

Same as TR-INGEST-001 output schema

• Output Schema:

Same as Input Schema (structure unchanged, records filtered)

• Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame for filtering operations

2. For each column in customer dataset:

• Filter out rows where column value equals string "Null" (case-sensitive exact match)

• Filter out rows where column value is NULL (actual null value)

3. Apply combined filter condition using AND logic across all columns

4. Count removed records: removed_count = original_count - filtered_count

5. Convert filtered Spark DataFrame back to DynamicFrame

6. Store result in `customer_clean_nulls_df`

• Validation Rules:

• All columns must be checked for null conditions

• String comparison for "Null" is case-sensitive

- Both string "Null" and actual NULL must be removed

- Error Handling and Logging:
- Log start of null removal with original record count
- Log count of removed records
- If all records are removed: Log warning "All customer records removed due to null values" but continue processing (do not fail)
- If no null values found: Log info "No null values found in customer dataset"
- Log completion with final record count
- All logs written to CloudWatch Logs

- Runtime Parameters:
- `--NULL_STRING_VALUE`: "Null" (default, configurable for different null representations)

- Operational Notes:
- Use Spark DataFrame filter operations for performance
- Consider caching DataFrame if used multiple times downstream
- Empty result set is valid and should not fail the job
-

TR-CLEAN-002: Remove Null Values from Order Dataset

- Related Functional Requirement ID(s): FR-CLEAN-001

- Objective:

Remove all records from order dataset that contain string literal "Null" or actual NULL values in any column.

- Source Details:
- Source Type: In-memory Glue DynamicFrame
- Source Name: order_raw_df (from TR-INGEST-002)

- Target Details:
- Target Type: In-memory Glue DynamicFrame
- Target Name: order_clean_nulls_df

- Input Schema:

Same as TR-INGEST-002 output schema

- Output Schema:

Same as Input Schema (structure unchanged, records filtered)

• Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame for filtering operations

2. For each column in order dataset:

• Filter out rows where column value equals string "Null" (case-sensitive exact match)

• Filter out rows where column value is NULL (actual null value)

3. Apply combined filter condition using AND logic across all columns

4. Count removed records: removed_count = original_count - filtered_count

5. Convert filtered Spark DataFrame back to DynamicFrame

6. Store result in `order_clean_nulls_df`

• Validation Rules:

• All columns must be checked for null conditions

• String comparison for "Null" is case-sensitive

• Both string "Null" and actual NULL must be removed

• Error Handling and Logging:

• Log start of null removal with original record count

• Log count of removed records

• If all records are removed: Log warning "All order records removed due to null values" but continue processing (do not fail)

• If no null values found: Log info "No null values found in order dataset"

• Log completion with final record count

• All logs written to CloudWatch Logs

• Runtime Parameters:

• `--NULL_STRING_VALUE`: "Null" (default, configurable for different null representations)

• Operational Notes:

• Use Spark DataFrame filter operations for performance

• Consider caching DataFrame if used multiple times downstream

• Empty result set is valid and should not fail the job

•

TR-CLEAN-003: Remove Duplicate Records from Customer Dataset

• Related Functional Requirement ID(s): FR-CLEAN-002

• Objective:

Remove duplicate records from customer dataset based on exact row matching across all columns, retaining only one instance of each unique record.

- Source Details:
- Source Type: In-memory Glue DynamicFrame
- Source Name: customer_clean_nulls_df (from TR-CLEAN-001)

- Target Details:
- Target Type: In-memory Glue DynamicFrame
- Target Name: customer_clean_df

- Input Schema:

Same as TR-CLEAN-001 output schema

- Output Schema:

Same as Input Schema (structure unchanged, duplicate records removed)

- Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame
2. Apply `dropDuplicates()` operation without specifying subset (all columns used for comparison)
3. Count removed duplicates: duplicate_count = original_count - deduplicated_count
4. Convert deduplicated Spark DataFrame back to DynamicFrame
5. Store result in `customer_clean_df`

- Validation Rules:
- Duplicate detection must compare all columns
- Exact match required across all column values
- First occurrence of duplicate is retained (Spark default behavior)

- Error Handling and Logging:
- Log start of deduplication with input record count
- Log count of removed duplicates
- If all records are duplicates (result is zero records): Log error "All customer records are duplicates, no unique records remain" and raise exception to fail job
- If no duplicates found: Log info "No duplicate records found in customer dataset"
- Log completion with final record count
- All logs written to CloudWatch Logs

- Runtime Parameters:

None specific to this operation

• Operational Notes:

• Spark `dropDuplicates()` is deterministic but order-dependent for tie-breaking

• Performance: Deduplication may trigger shuffle operation for large datasets

• Consider partitioning strategy if dataset is very large

•

TR-CLEAN-004: Remove Duplicate Records from Order Dataset

• Related Functional Requirement ID(s): FR-CLEAN-002

• Objective:

Remove duplicate records from order dataset based on exact row matching across all columns, retaining only one instance of each unique record.

• Source Details:

• Source Type: In-memory Glue DynamicFrame

• Source Name: order_clean_nulls_df (from TR-CLEAN-002)

• Target Details:

• Target Type: In-memory Glue DynamicFrame

• Target Name: order_clean_df

• Input Schema:

Same as TR-CLEAN-002 output schema

• Output Schema:

Same as Input Schema (structure unchanged, duplicate records removed)

• Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame

2. Apply `dropDuplicates()` operation without specifying subset (all columns used for comparison)

3. Count removed duplicates: duplicate_count = original_count - deduplicated_count

4. Convert deduplicated Spark DataFrame back to DynamicFrame

5. Store result in `order_clean_df`

• Validation Rules:

• Duplicate detection must compare all columns

• Exact match required across all column values

• First occurrence of duplicate is retained (Spark default behavior)

- Error Handling and Logging:
- Log start of deduplication with input record count
- Log count of removed duplicates
- If all records are duplicates (result is zero records): Log error "All order records are duplicates, no unique records remain" and raise exception to fail job
- If no duplicates found: Log info "No duplicate records found in order dataset"
- Log completion with final record count
- All logs written to CloudWatch Logs

- Runtime Parameters:

None specific to this operation

- Operational Notes:
- Spark `dropDuplicates()` is deterministic but order-dependent for tie-breaking
- Performance: Deduplication may trigger shuffle operation for large datasets
- Consider partitioning strategy if dataset is very large

-

TR-CATALOG-001: Write and Register Customer Table in Glue Data Catalog

- Related Functional Requirement ID(s): FR-CATALOG-001

- Objective:

Write cleaned customer data to S3 in Parquet format and register as a queryable table in AWS Glue Data Catalog.

- Source Details:
- Source Type: In-memory Glue DynamicFrame
- Source Name: customer_clean_df (from TR-CLEAN-003)

- Target Details:
- Target Type: S3 Parquet Files + Glue Catalog Table
- Target S3 Path: s3://adif-sdlc/catalog/sdlc_wizard/customer/
- Target Database: gen_ai_poc_databrickscoe
- Target Table: sdlc_wizard_customer
- File Format: Parquet
- Write Mode: Overwrite

- Input Schema:

Same as TR-CLEAN-003 output schema

• Output Schema:

Not enough information available in the FRD to derive complete schema details for sdlc_wizard_customer target table. Based on FRD, the following columns are present:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	(not specified)	(not specified)	(not specified)	
	Name	(not specified)	(not specified)	(not specified)	
	EmailId	(not specified)	(not specified)	(not specified)	
	Region	(not specified)	(not specified)	(not specified)	

• Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame if necessary

2. Write DataFrame to S3 using Parquet format:

• Target path: s3://adif-sdlc/catalog/sdlc_wizard/customer/

• Mode: overwrite

• Format: parquet

3. Use `glueContext.write_dynamic_frame.from_catalog()` or `glueContext.getSink()` to register table

4. Specify catalog parameters:

• database: gen_ai_poc_databrickscoe

• table_name: sdlc_wizard_customer

• transformation_ctx: "write_customer_catalog"

5. Update table statistics using `ANALYZE TABLE` command via Athena or Glue API (optional but recommended)

• Validation Rules:

• Target database `gen_ai_poc_databrickscoe` must exist before write operation

• S3 target path must be writable

• Schema must be compatible with Parquet format

• Table name must follow Glue naming conventions (lowercase, alphanumeric, underscores)

• Error Handling and Logging:

• Log start of write operation with target path and record count

• If database does not exist: Log error "Glue database gen_ai_poc_databrickscoe not found" and raise exception to fail job

• If S3 write fails: Log error with S3 path and exception details, raise exception to fail job, do not create catalog entry

• If catalog registration fails: Log error with table name and exception details, raise exception to fail job

• On successful write: Log "Customer table written to S3 and registered in catalog" with record count and S3 path

• All logs written to CloudWatch Logs

• Runtime Parameters:

• `--TARGET_CUSTOMER_S3_PATH`: s3://adif-sdlc/catalog/sdlc_wizard/customer/ (default)

• `--CATALOG_DATABASE`: gen_ai_poc_databrickscoe (default)

• `--CATALOG_TABLE_CUSTOMER`: sdlc_wizard_customer (default)

• `--WRITE_MODE`: overwrite (default)

• Operational Notes:

• Overwrite mode will replace existing data in S3 path

• Parquet format provides compression and columnar storage benefits

• Table is immediately queryable via Athena after registration

• Consider partitioning by Region if dataset grows large (not specified in FRD)

•

TR-CATALOG-002: Write and Register Order Table in Glue Data Catalog

• Related Functional Requirement ID(s): FR-CATALOG-002

• Objective:

Write cleaned order data to S3 in Parquet format and register as a queryable table in AWS Glue Data Catalog.

• Source Details:

• Source Type: In-memory Glue DynamicFrame

• Source Name: order_clean_df (from TR-CLEAN-004)

• Target Details:

• Target Type: S3 Parquet Files + Glue Catalog Table

• Target S3 Path: s3://adif-sdlc/catalog/sdlc_wizard/order/

• Target Database: gen_ai_poc_databrickscoe

• Target Table: sdlc_wizard_order

• File Format: Parquet

• Write Mode: Overwrite

• Input Schema:

Same as TR-CLEAN-004 output schema

• Output Schema:

Not enough information available in the FRD to derive complete schema details for sdlc_wizard_order target table. Based on FRD, the following columns are present:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	(not specified)	(not specified)	(not specified)	
	ItemName	(not specified)	(not specified)	(not specified)	
	PricePerUnit	(not specified)	(not specified)	(not specified)	
	Qty	(not specified)	(not specified)	(not specified)	
	Date	(not specified)	(not specified)	(not specified)	
	CustId	(not specified)	(not specified)	(not specified)	

• Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame if necessary

2. Write DataFrame to S3 using Parquet format:

• Target path: s3://adif-sdlc/catalog/sdlc_wizard/order/

• Mode: overwrite

• Format: parquet

3. Use `glueContext.write_dynamic_frame.from_catalog()` or `glueContext.getSink()` to register table

4. Specify catalog parameters:

• database: gen_ai_poc_databrickscoe

• table_name: sdlc_wizard_order

• transformation_ctx: "write_order_catalog"

5. Update table statistics using `ANALYZE TABLE` command via Athena or Glue API (optional but recommended)

• Validation Rules:

• Target database `gen_ai_poc_databrickscoe` must exist before write operation

• S3 target path must be writable

• Schema must be compatible with Parquet format

• Table name must follow Glue naming conventions (lowercase, alphanumeric, underscores)

• Error Handling and Logging:

• Log start of write operation with target path and record count

• If database does not exist: Log error "Glue database gen_ai_poc_databrickscoe not found" and raise exception to fail job

• If S3 write fails: Log error with S3 path and exception details, raise exception to fail job, do not create catalog entry

• If catalog registration fails: Log error with table name and exception details, raise exception to fail job

• On successful write: Log "Order table written to S3 and registered in catalog" with record count and S3 path

• All logs written to CloudWatch Logs

• Runtime Parameters:

• `--TARGET_ORDER_S3_PATH`: s3://adif-sdlc/catalog/sdlc_wizard/order/ (default)

• `--CATALOG_DATABASE`: gen_ai_poc_databrickscoe (default)

• `--CATALOG_TABLE_ORDER`: sdlc_wizard_order (default)

• `--WRITE_MODE`: overwrite (default)

• Operational Notes:

• Overwrite mode will replace existing data in S3 path

• Parquet format provides compression and columnar storage benefits

• Table is immediately queryable via Athena after registration

• Consider partitioning by Date if dataset grows large (not specified in FRD)

•

TR-SCD2-001: Implement SCD Type 2 for Order Summary Using Hudi

• Related Functional Requirement ID(s): FR-SCD2-001, FR-CATALOG-003

• Objective:

Join customer and order datasets, implement SCD Type 2 logic to track historical changes, and write results to Apache Hudi table on S3 with Glue Catalog registration.

• Source Details:

• Source Type 1: In-memory Glue DynamicFrame

• Source Name 1: customer_clean_df (from TR-CLEAN-003)

• Source Type 2: In-memory Glue DynamicFrame

• Source Name 2: order_clean_df (from TR-CLEAN-004)

• Source Type 3: Existing Hudi Table (if present)

• Source Name 3: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Target Details:

• Target Type: Apache Hudi Table (COW) + Glue Catalog Table

• Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Target Database: gen_ai_poc_databrickscoe

• Target Table: ordersummary

• Hudi Table Type: COPY_ON_WRITE

• Write Mode: Upsert

• Input Schema:

Customer (from customer_clean_df):

• CustId, Name, EmailId, Region (datatypes not specified)

Order (from order_clean_df):

• OrderId, ItemName, PricePerUnit, Qty, Date, CustId (datatypes not specified)

• Output Schema:

Not enough information available in the FRD to derive complete schema details for ordersummary target table. Based on FRD, the following columns are expected (joined customer + order columns plus SCD2 attributes):

Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping
----------	----------	----------	----------	----------
CustId	(not specified)	(not specified)	(not specified)	Direct map from customer.CustId and order.CustId
Name	(not specified)	(not specified)	(not specified)	Direct map from customer.Name
EmailId	(not specified)	(not specified)	(not specified)	Direct map from customer.EmailId
Region	(not specified)	(not specified)	(not specified)	Direct map from customer.Region
OrderId	(not specified)	(not specified)	(not specified)	Direct map from order.OrderId
ItemName	(not specified)	(not specified)	(not specified)	Direct map from order.ItemName
PricePerUnit	(not specified)	(not specified)	(not specified)	Direct map from order.PricePerUnit
Qty	(not specified)	(not specified)	(not specified)	Direct map from order.Qty
Date	(not specified)	(not specified)	(not specified)	Direct map from order.Date
IsActive	Boolean	N/A	No	System-generated: true for current records, false for historical
StartDate	Date	N/A	No	System-generated: current date for new/updated records
EndDate	Date	N/A	Yes	System-generated: null for active records, current date for closed record
OpTs	Timestamp	N/A	No	System-generated: current timestamp at processing time

• Processing / Transformation Logic:

• Step 1: Join Customer and Order Data

1. Convert customer_clean_df and order_clean_df to Spark DataFrames

2. Perform inner join on CustId:

```

joined_df = customer_df.join(order_df, customer_df.CustId == order_df.CustId, "inner")
```

```

3. Select all columns from both datasets (handle duplicate CustId column)

4. Result: joined_df with combined customer and order attributes

• Step 2: Add SCD2 Attributes to New Records

1. Add columns to joined_df:

• IsActive = true (literal boolean)

• StartDate = current_date() (Spark SQL function)

• EndDate = null (literal null)

• OpTs = current_timestamp() (Spark SQL function)

2. Result: new_records_df with SCD2 attributes

• Step 3: Read Existing Hudi Table (if exists)

1. Check if Hudi table exists at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

2. If exists:

• Read Hudi table into Spark DataFrame: existing_records_df

• Filter for active records: existing_active_df = existing_records_df.filter(col("IsActive") == true)

3. If not exists:

• Create empty DataFrame with ordersummary schema

• existing_active_df = empty DataFrame

• Step 4: Detect Changes (SCD2 Logic)

1. Define business key for comparison (assumed to be OrderId based on FRD context)

2. For each record in new_records_df:

• Look up matching record in existing_active_df by business key (OrderId)

• If no match found: Mark as INSERT (new record)

• If match found:

• Compare all attribute columns (exclude SCD2 columns)

• If any attribute differs: Mark as UPDATE (changed record)

• If all attributes match: Mark as NO_CHANGE (unchanged record)

• Step 5: Process INSERT Records

1. Filter new_records_df for INSERT records

2. Write to Hudi table with operation type = "insert"

3. These records will have IsActive=true, StartDate=current_date, EndDate=null, OpTs=current_timestamp

• Step 6: Process UPDATE Records
```

1. Filter new_records_df for UPDATE records

2. For each UPDATE record:

• Close existing active record:

• Set IsActive = false

• Set EndDate = current_date()

• Keep original StartDate and OpTs

• Insert new version:

• Set IsActive = true

• Set StartDate = current_date()

• Set EndDate = null

• Set OpTs = current_timestamp()

3. Combine closed records and new versions into update_df

4. Write to Hudi table with operation type = "upsert"

• Step 7: Process NO_CHANGE Records

1. No action required for unchanged records

2. Log count of unchanged records for monitoring

• Step 8: Write to Hudi Table

1. Configure Hudi write options:

• hoodie.table.name = "ordersummary"

• hoodie.datasource.write.recordkey.field = "OrderId" (assumed business key)

• hoodie.datasource.write.precombine.field = "OpTs"

• hoodie.datasource.write.partitionpath.field = "" (no partitioning specified in FRD)

• hoodie.datasource.write.table.type = "COPY_ON_WRITE"

• hoodie.datasource.write.operation = "upsert"

• hoodie.datasource.hive_sync.enable = "true"

• hoodie.datasource.hive_sync.database = "gen_ai_poc_databrickscoe"

• hoodie.datasource.hive_sync.table = "ordersummary"

• hoodie.datasource.hive_sync.partition_extractor_class = "
org.apache.hudi.hive.NonPartitionedExtractor"

2. Write combined DataFrame (inserts + updates) to Hudi table at
s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

3. Hudi will handle upsert logic based on recordkey and precombine field

• Step 9: Register in Glue Catalog

1. Hudi Hive sync will automatically register/update table in Glue Catalog

2. Verify table registration:

• Database: gen_ai_poc_databrickscoe

• Table: ordersummary

• Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Format: Hudi

3. Update table properties for Athena compatibility:

• Set table property: 'spark.sql.sources.provider' = 'hudi'

• Validation Rules:

• CustId must be present in both customer and order datasets for join

• Business key (OrderId) must be unique within new_records_df

• IsActive must be boolean type

• StartDate and EndDate must be date type

• OpTs must be timestamp type

• Hudi recordkey field must not contain nulls

• Hudi precombine field must be comparable (timestamp)

• Error Handling and Logging:

• Log start of SCD2 processing with input record counts

• If join produces zero records: Log warning "Join between customer and order datasets produced no records" and create empty Hudi table

• If CustId is missing in any record: Exclude from join and log warning with count

• If Hudi write fails: Log error with exception details and raise exception to fail job

• If Hudi table cannot be read (first run): Log info "No existing Hudi table found, treating all records as inserts"

• If Glue Catalog registration fails: Log error but do not fail job (table exists in S3)

• Log counts: total records, inserts, updates, no-changes, closed records

• Log completion with final active record count

• All logs written to CloudWatch Logs

• Runtime Parameters:

• `--HUDI_TABLE_NAME`: ordersummary (default)

• `--HUDI_RECORDKEY_FIELD`: OrderId (default, may need adjustment based on actual business key)

• `--HUDI_PRECOMBINE_FIELD`: OpTs (default)

• `--HUDI_TABLE_TYPE`: COPY_ON_WRITE (default)

• `--TARGET_ORDERSUMMARY_S3_PATH`: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (default)

• `--CATALOG_DATABASE`: gen_ai_poc_databrickscoe (default)

• `--CATALOG_TABLE_ORDERSUMMARY`: ordersummary (default)

• Operational Notes:

• First execution will treat all records as inserts

• Subsequent executions will perform change detection and SCD2 logic

• Hudi COW table type is recommended for Athena query performance

• Hudi compaction should be scheduled periodically for optimal performance

• Business key assumption (OrderId) may need validation based on actual requirements

• If multiple orders can have same OrderId (unlikely), composite key may be needed

• Consider partitioning by Date or Region for large datasets (not specified in FRD)

•

TR-INCR-001: Incremental Processing Based on Customer Updates

• Related Functional Requirement ID(s): FR-INCR-001

• Objective:

Detect changes in customer data and trigger SCD Type 2 updates in ordersummary table for affected customer records.

• Source Details:

• Source Type 1: S3 CSV Files (new/updated customer data)

• Source Path 1: s3://adif-sdlc/sdlc_wizard/customerdata/

• Source Type 2: Existing Hudi Table

• Source Path 2: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Target Details:

• Target Type: Apache Hudi Table (COW)

• Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Write Mode: Upsert (incremental update)

• Input Schema:

Customer (from s3://adif-sdlc/sdlc_wizard/customerdata/):

• CustId, Name, EmailId, Region (datatypes not specified)

Existing ordersummary (from Hudi table):

• All columns from TR-SCD2-001 output schema

• Output Schema:

Same as TR-SCD2-001 output schema (ordersummary table with updated SCD2 attributes)

• Processing / Transformation Logic:

• Step 1: Load New Customer Data

1. Read customer CSV files from s3://adif-sdlc/sdlc_wizard/customerdata/ (same as TR-INGEST-001)

2. Apply cleaning operations (null removal, deduplication) as per TR-CLEAN-001 and TR-CLEAN-003

3. Result: new_customer_df

• Step 2: Load Previous Customer Data (for change detection)

1. Option A: Read from previous customer catalog table (gen_ai_poc_databrickscoe.sdlc_wizard_customer)

2. Option B: Maintain separate customer history table (not specified in FRD)

3. Assumption: Use catalog table from previous run as baseline

4. Read previous customer data: previous_customer_df = read from catalog table

5. If previous data not available (first run): Treat all customers as new

• Step 3: Detect Changed Customers

1. Perform full outer join between new_customer_df and previous_customer_df on CustId

2. Identify changed customers:

• New customers: CustId exists in new_customer_df but not in previous_customer_df

• Updated customers: CustId exists in both, but any attribute (Name, EmailId, Region) differs

• Deleted customers: CustId exists in previous_customer_df but not in new_customer_df (not specified in FRD, log only)

3. Filter for new and updated customers: changed_customers_df

4. Extract list of changed CustIds: changed_custid_list

• Step 4: Identify Affected Order Summary Records

1. Read existing ordersummary Hudi table: ordersummary_df

2. Filter for active records: active_ordersummary_df = ordersummary_df.filter(col("IsActive") == true)

3. Filter for records matching changed CustIds:

```
affected_records_df = active_ordersummary_df.filter(col("CustId").isin(changed_custid_list))
```

4. Result: affected_records_df contains all active order summary records for changed customers

• Step 5: Close Affected Active Records

1. For each record in affected_records_df:

• Set IsActive = false

• Set EndDate = current_date()

• Keep original StartDate and OpTs unchanged

2. Result: closed_records_df

• Step 6: Create New Active Records with Updated Customer Data

1. Join affected_records_df with changed_customers_df on CustId

2. Replace customer attributes (Name, EmailId, Region) with new values from changed_customers_df

3. Keep order attributes (OrderId, ItemName, PricePerUnit, Qty, Date) unchanged

4. Set SCD2 attributes:

• IsActive = true

• StartDate = current_date()

• EndDate = null

• OpTs = current_timestamp()

5. Result: new_active_records_df

• Step 7: Write Updates to Hudi Table

1. Combine closed_records_df and new_active_records_df into update_df

2. Configure Hudi write options (same as TR-SCD2-001):

• hoodie.datasource.write.operation = "upsert"

• Other Hudi configurations as per TR-SCD2-001

3. Write update_df to Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

4. Hudi will handle upsert logic based on recordkey (OrderId) and precombine field (OpTs)

• Step 8: Update Customer Catalog Table

1. Write new_customer_df to customer catalog table (overwrite mode) as per TR-CATALOG-001

2. This establishes new baseline for next incremental run

• Validation Rules:

• Changed CustIds must exist in ordersummary table

• Customer attributes must be non-null after change detection

• SCD2 attributes must be properly set for closed and new records

• Hudi upsert must not create duplicate active records for same OrderId

• Error Handling and Logging:

• Log start of incremental processing with timestamp

• If no customer changes detected: Log info "No customer changes detected, skipping incremental processing" and exit successfully

• If previous customer data not available: Log warning "No previous customer data found, treating as initial load" and skip change detection

• If ordersummary table not found: Log error "ordersummary table not found, cannot perform incremental update" and raise exception to fail job

• If no affected records found for changed customers: Log info "No order summary records affected by customer changes"

• Log counts: changed customers, affected order summary records, closed records, new active records

• If Hudi upsert fails: Log error with exception details and raise exception to fail job

• Log completion with final record counts

• All logs written to CloudWatch Logs

• Runtime Parameters:

• `--ENABLE_INCREMENTAL_PROCESSING`: true (default, set to false to skip incremental logic)

• `--PREVIOUS_CUSTOMER_TABLE`: gen_ai_poc_databrickscoe.sdlc_wizard_customer (default)

• Other parameters inherited from TR-SCD2-001

• Operational Notes:

• Incremental processing assumes customer catalog table is updated after each run

• Change detection uses full comparison (no timestamp-based CDC)

• Performance: Incremental processing is more efficient than full reprocessing for large datasets

• First run should skip incremental logic and perform full SCD2 processing (TR-SCD2-001)

• Consider implementing timestamp-based change detection if source provides last_modified_date

• Deleted customers are logged but not processed (no requirement in FRD)

•

TR-AGG-001: Generate Customer Aggregate Spend Table

• Related Functional Requirement ID(s): FR-AGG-001, FR-CATALOG-004

• Objective:

Join customer and order datasets, calculate total spending per customer per date, and write results to S3 with Glue Catalog registration.

• Source Details:

• Source Type 1: In-memory Glue DynamicFrame

• Source Name 1: customer_clean_df (from TR-CLEAN-003)

• Source Type 2: In-memory Glue DynamicFrame

• Source Name 2: order_clean_df (from TR-CLEAN-004)

• Target Details:

• Target Type: S3 Parquet Files + Glue Catalog Table

• Target S3 Path: s3://adif-sdlc/analytics/customeraggregatespend/

• Target Database: gen_ai_poc_databrickscoe

• Target Table: customeraggregatespend

• File Format: Parquet

• Write Mode: Overwrite

• Input Schema:

Customer (from customer_clean_df):

• CustId, Name, EmailId, Region (datatypes not specified)

Order (from order_clean_df):

• OrderId, ItemName, PricePerUnit, Qty, Date, CustId (datatypes not specified)

• Output Schema:

Not enough information available in the FRD to derive complete schema details for customeraggregatespend target table. Based on FRD, the following columns are expected:

nn Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping
------	----------	----------	----------	----------
e	(not specified)	(not specified)	(not specified)	Direct map from customer.Name
Amount	(not specified, assumed Decimal)	(not specified)	(not specified)	Derived: sum(PricePerUnit * Qty) grouped by Name and
	(not specified)	(not specified)	(not specified)	Direct map from order.Date

• Processing / Transformation Logic:

• Step 1: Join Customer and Order Data

1. Convert customer_clean_df and order_clean_df to Spark DataFrames

2. Perform inner join on CustId:

```
joined_df = customer_df.join(order_df, customer_df.CustId == order_df.CustId, "inner")
```

3. Select required columns: Name (from customer), PricePerUnit, Qty, Date (from order)

• Step 2: Calculate TotalAmount per Order

1. Add calculated column to joined_df:

```
joined_df = joined_df.withColumn("TotalAmount", col("PricePerUnit") col("Qty"))
```

2. Handle non-numeric values:

• If PricePerUnit or Qty cannot be cast to numeric type, exclude record and log error

• Use try-cast or filter to ensure numeric types

• Step 3: Aggregate by Customer Name and Date

1. Group by Name and Date:

```
agg_df = joined_df.groupBy("Name", "Date").agg(sum("TotalAmount").alias("TotalAmount"))
```

2. Result: agg_df with columns Name, TotalAmount, Date

• Step 4: Select Final Columns

1. Select columns in specified order: Name, TotalAmount, Date

2. Ensure column order matches output schema

• Step 5: Write to S3 and Register in Catalog

1. Write agg_df to S3 using Parquet format:

• Target path: s3://adif-sdlc/analytics/customeraggregatespend/

• Mode: overwrite

• Format: parquet

2. Register table in Glue Catalog:

• Database: gen_ai_poc_databrickscoe

• Table name: customeraggregatespend

• Location: s3://adif-sdlc/analytics/customeraggregatespend/

3. Update table statistics (optional but recommended)

• Validation Rules:

• CustId must be present in both customer and order datasets for join

• PricePerUnit and Qty must be numeric types

• TotalAmount calculation must not produce null values

• Name and Date must not be null in final output

• Aggregation must produce one record per unique (Name, Date) combination

• Error Handling and Logging:

• Log start of aggregation with input record counts

• If join produces zero records: Log warning "Join between customer and order datasets produced no records" and create empty table

• If PricePerUnit or Qty are non-numeric: Exclude those records and log error with count

• If aggregation produces zero records: Log warning "Aggregation produced no records" and create empty table

• If S3 write fails: Log error with S3 path and exception details, raise exception to fail job, do not create catalog entry

• If catalog registration fails: Log error with table name and exception details, raise exception to fail job

• Log successful write with record count and S3 path

• All logs written to CloudWatch Logs

• Runtime Parameters:

• `--TARGET_AGGREGATE_S3_PATH`: s3://adif-sdlc/analytics/customeraggregatespend/ (default)

• `--CATALOG_DATABASE`: gen_ai_poc_databrickscoe (default)

• `--CATALOG_TABLE_AGGREGATE`: customeraggregatespend (default)

• `--WRITE_MODE`: overwrite (default)

• Operational Notes:

• Overwrite mode will replace existing aggregate data

• Consider incremental aggregation for large datasets (not specified in FRD)

• Parquet format provides efficient storage for analytical queries

• Table is immediately queryable via Athena after registration

• Consider partitioning by Date for large datasets (not specified in FRD)

• TotalAmount datatype should be Decimal to avoid precision loss

•

3. Runtime Strategy Notes

Authoring Language

• Primary Language: PySpark (Python with Spark APIs)

• AWS Glue Version: AWS Glue 3.0 or higher

• Python Version: Python 3.7 or higher (Glue 3.0 default)

• Spark Version: Spark 3.1 or higher (Glue 3.0 default)

Execution Strategy

• Job Type: AWS Glue ETL Job (Spark)

- Worker Type: G.1X or G.2X (Standard workers)
- Number of Workers: 2-10 (configurable based on data volume)
- Max Capacity: Not applicable (using worker type and count)
- Timeout: 2880 minutes (48 hours, configurable)
- Max Retries: 0 (default, configurable)
- Job Bookmark: Disabled (full refresh processing, enable for incremental if CDC available)
- Execution Mode: Synchronous (single job execution)

Glue Job Type Expectations

- ETL Job: Standard Glue Spark job for data transformation
- Script Location: S3 path for Python script (e.g., s3://adif-sdlc/scripts/customer_order_etl.py)
- Temporary Directory: S3 path for Spark temporary files (e.g., s3://adif-sdlc/temp/)
- Spark UI Logs: Enabled for debugging (optional)
- Continuous Logging: Enabled for real-time CloudWatch log streaming
- Job Parameters: Pass runtime parameters as --key value pairs
- IAM Role: Glue service role with permissions for S3, Glue Catalog, CloudWatch Logs
- VPC Configuration: Not required unless accessing VPC resources (not specified in FRD)
- Security Configuration: Optional for encryption at rest and in transit
-

4. Data Design Details

Source Paths / Source Systems

urce ID	Source Type	Source Path	Format	Description
--------	----------	----------	--------	----------
C-001	S3 CSV	s3://adif-sdlc/sdlc_wizard/customerdata/	CSV	Customer master data files
C-002	S3 CSV	s3://adif-sdlc/sdlc_wizard/orderdata/	CSV	Order transaction data files
C-003	Hudi Table	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi COW	Existing order summary table (for incremental processi

Target Paths / Target Tables

get ID	Target Type	Target Path	Format	Description	Write M
-------	----------	----------	--------	----------	----------
T-001	S3 Parquet + Catalog	s3://adif-sdlc/catalog/sdlc_wizard/customer/	Parquet	Cleaned customer catalog table	Overwri
T-002	S3 Parquet + Catalog	s3://adif-sdlc/catalog/sdlc_wizard/order/	Parquet	Cleaned order catalog table	Overwri
T-003	Hudi Table + Catalog	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi COW	Order summary with SCD2 history	Upsert
T-004	S3 Parquet + Catalog	s3://adif-sdlc/analytics/customeraggregatespend/	Parquet	Customer aggregate spend analytics	Overwri

Catalog Registration Expectations

	Database	Table Name	Location	Format	Partitioned	Athena C
	----------	------------	----------	--------	-------------	----------
	gen_ai_poc_databrickscoe	sdlc_wizard_customer	s3://adif-sdlc/catalog/sdlc_wizard/customer/	Parquet	No	Yes
	gen_ai_poc_databrickscoe	sdlc_wizard_order	s3://adif-sdlc/catalog/sdlc_wizard/order/	Parquet	No	Yes
	gen_ai_poc_databrickscoe	ordersummary	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi	No	Yes (wit
ate	gen_ai_poc_databrickscoe	customeraggregatespend	s3://adif-sdlc/analytics/customeraggregatespend/	Parquet	No	Yes

Partitioning Expectations

- Customer Catalog Table: No partitioning specified in FRD
- Order Catalog Table: No partitioning specified in FRD (consider partitioning by Date for large datasets)
- Order Summary Hudi Table: No partitioning specified in FRD (consider partitioning by Date or Region for large datasets)
- Customer Aggregate Table: No partitioning specified in FRD (consider partitioning by Date for large datasets)

File Formats / Table Formats

	Table	Storage Format	Compression	Columnar	Notes	
	-------	----------	----------	----------	-------	
	Customer Catalog	Parquet	Snappy (default)	Yes	Efficient for analytical queries	
	Order Catalog	Parquet	Snappy (default)	Yes	Efficient for analytical queries	
	Order Summary	Hudi COW	Parquet (base)	Yes	Supports ACID transactions, time travel	
	Customer Aggregate	Parquet	Snappy (default)	Yes	Optimized for BI tool queries	

Update / Overwrite / Append Behavior

	Target Table	Write Mode	Behavior	Rationale	
	----------	----------	----------	----------	
	sdlc_wizard_customer	Overwrite	Replace all data	Full refresh of customer master data	
	sdlc_wizard_order	Overwrite	Replace all data	Full refresh of order transaction data	
	ordersummary	Upsert	Insert new, update existing	SCD2 requires upsert to maintain history	
	customeraggregatespend	Overwrite	Replace all data	Full recalculation of aggregates	

- Note: No staging tables, intermediate layers, or derived sources are introduced beyond what is explicitly stated in the FRD.
-

5. Processing Details

Data Quality Rules

Rule ID	Rule Type	Description	Applied To	Action on Violation
---	---	---	---	---
DQ-001	Null Check	Remove records with string "Null" (case-sensitive)	Customer, Order	Exclude record, log count
DQ-002	Null Check	Remove records with actual NULL values	Customer, Order	Exclude record, log count
DQ-003	Duplicate Check	Remove exact duplicate rows (all columns)	Customer, Order	Retain first occurrence, log count
DQ-004	Type Validation	Ensure PricePerUnit and Qty are numeric	Order (for aggregation)	Exclude record, log error
DQ-005	Key Validation	Ensure CustId is not null	Customer, Order	Exclude from join, log warning

Deduplication Rules

Dataset	Deduplication Key	Method	Retention Logic
---	---	---	---
Customer	All columns	Exact match	Retain first occurrence (Spark default)
Order	All columns	Exact match	Retain first occurrence (Spark default)

Join Rules

Join ID	Left Dataset	Right Dataset	Join Type	Join Key	Null Handling
---	---	---	---	---	---
JOIN-001	Customer	Order	Inner	CustId	Exclude records with null CustId
JOIN-002	Customer	Order (for aggregation)	Inner	CustId	Exclude records with null CustId
JOIN-003	New Customer	Previous Customer	Full Outer	CustId	Used for change detection
JOIN-004	Affected Orders	Changed Customers	Inner	CustId	Update customer attributes

Aggregation Rules

on ID	Group By Columns	Aggregate Function	Aggregate Column	Output Column	Null Handling
---	---	---	---	---	---
	Name, Date	SUM	TotalAmount (PricePerUnit * Qty)	TotalAmount	Exclude records with null or non-num

SCD / History Tracking Rules

ype	Business Key	Change Detection Columns	Active Flag	Start Date	End Date	Operation Tim
-	---	---	---	---	---	---
ype 2	OrderId (assumed)	All customer and order attributes	IsActive (Boolean)	StartDate (Date)	EndDate (Date, nullable)	OpTs (Timest

• SCD Logic:

• Insert: New records get IsActive=true, StartDate=current_date, EndDate=null, OpTs=current_timestamp

• Update: Existing active record gets IsActive=false, EndDate=current_date; New version gets IsActive=true, StartDate=current_date, EndDate=null, OpTs=current_timestamp

• No Change: No action, existing active record remains unchanged

• Incremental Update: Customer changes trigger closure of affected active records and creation of new versions with updated customer attributes

Null Handling

	Null Type	Detection Method	Action	Applied To	
	---	---	---	---	
	String "Null"	Exact string match (case-sensitive)	Remove record	Customer, Order (raw data)	
	Actual NULL	IS NULL check	Remove record	Customer, Order (raw data)	
	NULL in join key	IS NULL check	Exclude from join	Customer, Order (join operations)	
	NULL in numeric fields	Type casting failure	Exclude from calculation, log error	PricePerUnit, Qty (aggregation)	

Type Casting Expectations

	Column	Source Type	Target Type	Casting Method	Error Handling	
	---	---	---	---	---	
	PricePerUnit	(not specified)	Decimal	Cast or try_cast	Exclude record if cast fails, log error	
	Qty	(not specified)	Integer or Decimal	Cast or try_cast	Exclude record if cast fails, log error	
	Date	(not specified)	Date	to_date() with format	Use default format or fail if unparseable	
	IsActive	N/A	Boolean	Literal assignment	N/A	
	StartDate	N/A	Date	current_date()	N/A	
	EndDate	N/A	Date	current_date() or null	N/A	
	OpTs	N/A	Timestamp	current_timestamp()	N/A	

•

6. Traceability Matrix

	Original Source	Technical Source
	---	---
	s3://adif-sdlc/sdlc_wizard/customerdata/	s3://adif-sdlc/sdlc_wizard/customerdata/
	s3://adif-sdlc/sdlc_wizard/orderdata/	s3://adif-sdlc/sdlc_wizard/orderdata/
002	customer_raw_df, order_raw_df	customer_raw_df, order_raw_df
004	customer_clean_nulls_df, order_clean_nulls_df	customer_clean_nulls_df, order_clean_nulls_df
	customer_clean_df	customer_clean_df
	order_clean_df	order_clean_df
	customer_clean_df, order_clean_df, existing ordersummary	customer_clean_df, order_clean_df, s3://adif-sdlc/curated/sdlc_wizard/ord
	s3://adif-sdlc/sdlc_wizard/customerdata/, existing ordersummary	s3://adif-sdlc/sdlc_wizard/customerdata/, s3://adif-sdlc/curated/sdlc_wizar
	customer_clean_df, order_clean_df	customer_clean_df, order_clean_df
	ordersummary Hudi table	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
	customeraggregatespend	s3://adif-sdlc/analytics/customeraggregatespend/

• Source Fidelity Verification:

• All sources in TRD match exactly with sources specified in FRD

• No intermediate sources, staging tables, or alternative sources have been introduced

• In-memory DataFrames (customer_clean_df, order_clean_df) are direct results of cleaning operations on original sources

• Hudi table source (s3://adif-sdlc/curated/sdlc_wizard/ordersummary/) is used only for incremental processing as specified in FRD

•

7. Additional Notes

Schema Details Limitation

Not enough information available in the FRD to derive complete schema details (data types, lengths, precision, scale, nullability) for the following sources and targets:

• Source: s3://adif-sdlc/sdlc_wizard/customerdata/ (Customer CSV)

• Source: s3://adif-sdlc/sdlc_wizard/orderdata/ (Order CSV)

• Target: gen_ai_poc_databrickscoe.sdlc_wizard_customer

• Target: gen_ai_poc_databrickscoe.sdlc_wizard_order

• Target: gen_ai_poc_databrickscoe.ordersummary

• Target: gen_ai_poc_databrickscoe.customeraggregatespend

• Recommendation: Schema details should be determined through one of the following methods:

1. Inspect sample CSV files from source S3 paths

2. Use AWS Glue Crawler to infer schema from source data

3. Obtain schema documentation from data source owners

4. Define schema explicitly in Glue job script with appropriate data types

Business Key Assumption

The TRD assumes OrderId as the business key (recordkey) for Hudi SCD2 implementation. This assumption is based on the context that OrderId is the primary identifier in the Order dataset. However, the FRD does not explicitly specify the business key. If OrderId is not unique or if a composite key is required (e.g., OrderId + CustId), the Hudi configuration must be adjusted accordingly.

Change Detection Mechanism

The incremental processing logic (TR-INCR-001) uses full dataset comparison for change detection, as no CDC (Change Data Capture) mechanism or timestamp-based tracking is specified in the FRD. For production environments with large datasets, consider implementing:

• Timestamp-based change detection (last_modified_date column)

• CDC using AWS DMS or similar tools

- Incremental file processing with file naming conventions

Hudi Table Compaction

Apache Hudi tables require periodic compaction to maintain query performance. The TRD does not include compaction logic as it is not specified in the FRD. For production deployments, schedule Hudi compaction jobs using:

- AWS Glue workflow with compaction job
- Hudi inline compaction (may impact write performance)
- Separate compaction job triggered by CloudWatch Events

Athena Query Compatibility

The ordersummary Hudi table is configured for Athena compatibility using COW (Copy-On-Write) table type. Ensure that:

- Athena workgroup has Hudi connector enabled
- Athena engine version supports Hudi format (Athena v2 or higher)
- Glue Catalog table properties include Hudi-specific metadata

Error Recovery and Idempotency

The TRD specifies error handling and logging but does not include detailed error recovery mechanisms. For production deployments, consider:

- Implementing checkpointing for long-running transformations
- Using Glue job bookmarks for incremental processing (if applicable)
- Designing idempotent operations to support job retries
- Implementing data quality checks with configurable thresholds
-

- End of Technical Requirements Document